1. Use MongoDB to implement the following DB operations

1. Create a database called 'vehicles' and write a MongoDB query to select database as"vehicles".

```
test> use vehicles
switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
admin      40.00 KiB
config     84.00 KiB
local      72.00 KiB
vehicles   80.00 KiB
```

3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
vehicles> db.createCollection("two_wheelers", {capped:true,size:100000})
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
vehicles> show collections
four_wheelers
two_wheelers
```

4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc,125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array,manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.two_wheelers.insertMany([
| {
| bike_name: "Honda Shine",
| model: "gear",
| category: "125cc",
| colors_available: ["red","black","blue"],
| manufacturer: "Honda",
| performance: 8,
| timestamp: new Date("2021-03-15"),
| price: 85000
| },
| {
| bike_name: "TVS Jupiter",
| model: "gearless",
| category: "125cc",
| colors_available: ["white","blue","black"],
| manufacturer: "TVS",
| performance: 7,
| timestamp: new Date("2020-06-20"),
| price: 78000
| },
| {
| bike_name: "Yamaha R15",
| model: "gear",
| category: "150cc",
| colors_available: ["sport red","black"],
| manufacturer: "Yamaha",
| performance: 9,
| timestamp: new Date("2022-01-10"),
| price: 180000
| },
| {
| bike_name: "Bajaj Pulsar",
| model: "gear",
| category: "200cc",
| colors_available: ["black","red"],
| manufacturer: "Bajaj",
| performance: 8,
| timestamp: new Date("2019-08-05"),
| price: 150000
| },
| {
| bike_name: "Hero Splendor",
| model: "gear",
| category: "100cc",
| colors_available: ["black","blue"],
| manufacturer: "Hero",
| performance: 7,
| timestamp: new Date("2018-11-11"),
| price: 72000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b166be156571f5c67c2907'),
    '1': ObjectId('69b166be156571f5c67c2908'),
    '2': ObjectId('69b166be156571f5c67c2909'),
    '3': ObjectId('69b166be156571f5c67c290a'),
    '4': ObjectId('69b166be156571f5c67c290b')
  }
}
```

5. Add 5 four-wheeler details to the collection named 'four_wheelers'. Each document consists of following fields as vehicle_name, model (commercial or own), category(car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp(date and year release) and price.

```
vehicles> db.four_wheelers.insertMany([
| {
| vehicle_name: "Maruti Swift",
| model: "own",
| category: "car",
| variants: ["vxi","zxi","petrol","diesel"],
| manufacturer: "Maruti",
| performance: 8,
| timestamp: new Date("2021-05-10"),
| price: 700000
| },
| {
| vehicle_name: "Tata Ace",
| model: "commercial",
| category: "mini truck",
| variants: ["diesel"],
| manufacturer: "Tata",
| performance: 7,
| timestamp: new Date("2019-02-12"),
| price: 450000
| },
| {
| vehicle_name: "Ashok Leyland Bus",
| model: "commercial",
| category: "bus",
| variants: ["diesel"],
| manufacturer: "Ashok Leyland",
| performance: 8,
| timestamp: new Date("2018-07-01"),
| price: 2500000
| },
| {
| vehicle_name: "Mahindra Bolero",
| model: "own",
| category: "car",
| variants: ["diesel"],
| manufacturer: "Mahindra",
| performance: 7,
| timestamp: new Date("2020-03-22"),
| price: 900000
| },
| {
| vehicle_name: "Volvo Container",
| model: "commercial",
| category: "containers",
| variants: ["diesel"],
| manufacturer: "Volvo",
| performance: 9,
| timestamp: new Date("2022-09-14"),
| price: 4500000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b166c6156571f5c67c290c'),
    '1': ObjectId('69b166c6156571f5c67c290d'),
    '2': ObjectId('69b166c6156571f5c67c290e'),
    '3': ObjectId('69b166c6156571f5c67c290f'),
    '4': ObjectId('69b166c6156571f5c67c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69b1650d71e2ec4e5d7c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2026-03-11T12:50:21.321Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b166be156571f5c67c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2021-03-15T00:00:00.000Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b166be156571f5c67c2908'),
    bike_name: 'TVS Jupiter',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'white', 'blue', 'black' ],
    manufacturer: 'TVS',
    performance: 7,
    timestamp: ISODate('2020-06-20T00:00:00.000Z'),
    price: 78000
  },
  {
    _id: ObjectId('69b166be156571f5c67c2909'),
    bike_name: 'Yamaha R15',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'sport red', 'black' ],
    manufacturer: 'Yamaha',
    performance: 9,
    timestamp: ISODate('2022-01-10T00:00:00.000Z'),
    price: 180000
  },
  {
    _id: ObjectId('69b166be156571f5c67c290a'),
    bike_name: 'Bajaj Pulsar',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'black', 'red' ],
    manufacturer: 'Bajaj',
    performance: 8,
    timestamp: ISODate('2019-08-05T00:00:00.000Z'),
    price: 150000
  },
  {
    _id: ObjectId('69b166be156571f5c67c290b'),
    bike_name: 'Hero Splendor',
    model: 'gear',
    category: '100cc',
    colors_available: [ 'black', 'blue' ],
    manufacturer: 'Hero',
    performance: 7,
    timestamp: ISODate('2018-11-11T00:00:00.000Z'),
    price: 72000
  }
]
```

```
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b1651771e2ec4e5d7c2908'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol', 'diesel' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2026-03-11T12:50:31.891Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290c'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol', 'diesel' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2021-05-10T00:00:00.000Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290d'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: ISODate('2019-02-12T00:00:00.000Z'),
    price: 450000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290e'),
    vehicle_name: 'Ashok Leyland Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 8,
    timestamp: ISODate('2018-07-01T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290f'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 7,
    timestamp: ISODate('2020-03-22T00:00:00.000Z'),
    price: 900000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c2910'),
    vehicle_name: 'Volvo Container',
    model: 'commercial',
    category: 'containers',
    variants: [ 'diesel' ],
    manufacturer: 'Volvo',
    performance: 9,
    timestamp: ISODate('2022-09-14T00:00:00.000Z'),
    price: 4500000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Honda Shine', price: 85000 },
  { bike_name: 'Honda Shine', price: 85000 },
  { bike_name: 'TVS Jupiter', price: 78000 },
  { bike_name: 'Yamaha R15', price: 180000 },
  { bike_name: 'Bajaj Pulsar', price: 150000 },
  { bike_name: 'Hero Splendor', price: 72000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _i_id:0})
[
  { vehicle_name: 'Maruti Swift', price: 700000 },
  { vehicle_name: 'Maruti Swift', price: 700000 },
  { vehicle_name: 'Tata Ace', price: 450000 },
  { vehicle_name: 'Ashok Leyland Bus', price: 2500000 },
  { vehicle_name: 'Mahindra Bolero', price: 900000 },
  { vehicle_name: 'Volvo Container', price: 4500000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0}vehicles> db.two_wheelers.find({manufacturer:"Honda"})
[
  {
    _id: ObjectId('69b1650d71e2ec4e5d7c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2026-03-11T12:50:21.321Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b166be156571f5c67c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2021-03-15T00:00:00.000Z'),
    price: 85000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
]
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b1651771e2ec4e5d7c2908'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol', 'diesel' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2026-03-11T12:50:31.891Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290c'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol', 'diesel' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2021-05-10T00:00:00.000Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290d'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: ISODate('2019-02-12T00:00:00.000Z'),
    price: 450000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290e'),
    vehicle_name: 'Ashok Leyland Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 8,
    timestamp: ISODate('2018-07-01T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c290f'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 7,
    timestamp: ISODate('2020-03-22T00:00:00.000Z'),
    price: 900000
  },
  {
    _id: ObjectId('69b166c6156571f5c67c2910'),
    vehicle_name: 'Volvo Container',
    model: 'commercial',
    category: 'containers',
    variants: [ 'diesel' ],
    manufacturer: 'Volvo',
    performance: 9,
    timestamp: ISODate('2022-09-14T00:00:00.000Z'),
    price: 4500000
  }
]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
vehicles> db.two_wheelers.find(
| {performance: {$gt:5}},
| {bike_name:1, category:1, manufacturer:1, _id:0}
| )
[
  { bike_name: 'Honda Shine', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'Honda Shine', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'TVS Jupiter', category: '125cc', manufacturer: 'TVS' },
  { bike_name: 'Yamaha R15', category: '150cc', manufacturer: 'Yamaha' },
  { bike_name: 'Bajaj Pulsar', category: '200cc', manufacturer: 'Bajaj' },
  { bike_name: 'Hero Splendor', category: '100cc', manufacturer: 'Hero' }
]
vehicles> db.four_wheelers.find(
| {performance: {$gt:5}},
| {vehicle_name:1, category:1, manufacturer:1, _id:0}
| )
[
  {
    vehicle_name: 'Maruti Swift',
    category: 'car',
    manufacturer: 'Maruti'
  },
  {
    vehicle_name: 'Maruti Swift',
    category: 'car',
    manufacturer: 'Maruti'
  },
  {
    vehicle_name: 'Tata Ace',
    category: 'mini truck',
    manufacturer: 'Tata'
  },
  {
    vehicle_name: 'Ashok Leyland Bus',
    category: 'bus',
    manufacturer: 'Ashok Leyland'
  },
  {
    vehicle_name: 'Mahindra Bolero',
    category: 'car',
    manufacturer: 'Mahindra'
  },
  {
    vehicle_name: 'Volvo Container',
    category: 'containers',
    manufacturer: 'Volvo'
  }
]
vehicles>
```

1. Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

```
test> use animal
switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
animal> show dbs
admin        40.00 KiB
config      108.00 KiB
local        72.00 KiB
vehicles    144.00 KiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
animal> db.createCollection("wild_animals", {capped:true, size:100000})
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
animal> show collections
domestic_animals
wild_animals
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.wild_animals.insertMany([
| {
| animal_name:"Lion",
| nature:"harm",
| favorite_foods:["meat","deer"],
| care_taker_name:"Ravi",
| life_span:14,
| timestamp:new Date("2022-01-10"),
| expenses:50000
| },
| {
| animal_name:"Tiger",
| nature:"harm",
| favorite_foods:["meat","rabbit"],
| care_taker_name:"Arjun",
| life_span:15,
| timestamp:new Date("2021-05-12"),
| expenses:55000
| },
| {
| animal_name:"Elephant",
| nature:"harmless",
| favorite_foods:["grass","banana"],
| care_taker_name:"Suresh",
| life_span:60,
| timestamp:new Date("2020-03-20"),
| expenses:70000
| },
| {
| animal_name:"Deer",
| nature:"harmless",
| favorite_foods:["grass","leaves"],
| care_taker_name:"Ravi",
| life_span:20,
| timestamp:new Date("2019-07-15"),
| expenses:20000
| },
| {
| animal_name:"Leopard",
| nature:"harm",
| favorite_foods:["meat","rabbit"],
| care_taker_name:"Kiran",
| life_span:12,
| timestamp:new Date("2023-02-18"),
| expenses:45000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b16ad69cb3b46b2a7c2987'),
    '1': ObjectId('69b16ad69cb3b46b2a7c2988'),
    '2': ObjectId('69b16ad69cb3b46b2a7c2989'),
    '3': ObjectId('69b16ad69cb3b46b2a7c298a'),
    '4': ObjectId('69b16ad69cb3b46b2a7c298b')
  }
}
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female),favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (inyears), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.domestic_animals.insertMany([
| {
| animal_name:"Dog",
| gender:"male",
| favorite_foods:["meat","rice"],
| animal_petname:"Tommy",
| life_span:12,
| timestamp:new Date("2022-04-10"),
| expenses:5000
| },
| {
| animal_name:"Cat",
| gender:"female",
| favorite_foods:["fish","milk"],
| animal_petname:"Kitty",
| life_span:10,
| timestamp:new Date("2021-06-15"),
| expenses:3000
| },
| {
| animal_name:"Cow",
| gender:"female",
| favorite_foods:["grass","hay"],
| animal_petname:"Lakshmi",
| life_span:18,
| timestamp:new Date("2020-01-20"),
| expenses:8000
| },
| {
| animal_name:"Goat",
| gender:"male",
| favorite_foods:["leaves","grass"],
| animal_petname:"Bunny",
| life_span:15,
| timestamp:new Date("2019-09-25"),
| expenses:2500
| },
| {
| animal_name:"Horse",
| gender:"male",
| favorite_foods:["grass","oats"],
| animal_petname:"Raja",
| life_span:25,
| timestamp:new Date("2023-03-05"),
| expenses:12000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b16ae29cb3b46b2a7c290c'),
    '1': ObjectId('69b16ae29cb3b46b2a7c290d'),
    '2': ObjectId('69b16ae29cb3b46b2a7c290e'),
    '3': ObjectId('69b16ae29cb3b46b2a7c290f'),
    '4': ObjectId('69b16ae29cb3b46b2a7c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
animal> db.wild_animals.find().pretty()
[
  {
    _id: ObjectId('69b16ad69cb3b46b2a7c2907'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2022-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b16ad69cb3b46b2a7c2908'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Arjun',
    life_span: 15,
    timestamp: ISODate('2021-05-12T00:00:00.000Z'),
    expenses: 55000
  },
  {
    _id: ObjectId('69b16ad69cb3b46b2a7c2909'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'banana' ],
    care_taker_name: 'Suresh',
    life_span: 60,
    timestamp: ISODate('2020-03-20T00:00:00.000Z'),
    expenses: 70000
  },
  {
    _id: ObjectId('69b16ad69cb3b46b2a7c290a'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Ravi',
    life_span: 20,
    timestamp: ISODate('2019-07-15T00:00:00.000Z'),
    expenses: 20000
  },
  {
    _id: ObjectId('69b16ad69cb3b46b2a7c290b'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Kiran',
    life_span: 12,
    timestamp: ISODate('2023-02-18T00:00:00.000Z'),
    expenses: 45000
  }
]
```

```
animal> db.domestic_animals.find().pretty()
[
  {
    _id: ObjectId('69b16ae29cb3b46b2a7c290c'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'rice' ],
    animal_petname: 'Tommy',
    life_span: 12,
    timestamp: ISODate('2022-04-10T00:00:00.000Z'),
    expenses: 5000
  },
  {
    _id: ObjectId('69b16ae29cb3b46b2a7c290d'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 10,
    timestamp: ISODate('2021-06-15T00:00:00.000Z'),
    expenses: 3000
  },
  {
    _id: ObjectId('69b16ae29cb3b46b2a7c290e'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'hay' ],
    animal_petname: 'Lakshmi',
    life_span: 18,
    timestamp: ISODate('2020-01-20T00:00:00.000Z'),
    expenses: 8000
  },
  {
    _id: ObjectId('69b16ae29cb3b46b2a7c290f'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [ 'leaves', 'grass' ],
    animal_petname: 'Bunny',
    life_span: 15,
    timestamp: ISODate('2019-09-25T00:00:00.000Z'),
    expenses: 2500
  },
  {
    _id: ObjectId('69b16ae29cb3b46b2a7c2910'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'grass', 'oats' ],
    animal_petname: 'Raja',
    life_span: 25,
    timestamp: ISODate('2023-03-05T00:00:00.000Z'),
    expenses: 12000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
animal> db.wild_animals.find({}, {animal_name:1, expenses:1,_id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 55000 },
  { animal_name: 'Elephant', expenses: 70000 },
  { animal_name: 'Deer', expenses: 20000 },
  { animal_name: 'Leopard', expenses: 45000 }
]
animal> db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Dog', expenses: 5000 },
  { animal_name: 'Cat', expenses: 3000 },
  { animal_name: 'Cow', expenses: 8000 },
  { animal_name: 'Goat', expenses: 2500 },
  { animal_name: 'Horse', expenses: 12000 }
]
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
animal> db.domestic_animals.find({life_span:10})
[
  {
    _id: ObjectId('69b16ae29cb3b46b2a7c290d'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 10,
    timestamp: ISODate('2021-06-15T00:00:00.000Z'),
    expenses: 3000
  }
]
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
animal> db.wild_animals.find({care_taker_name:"Ravi"})
[
  {
    _id: ObjectId('69b16ad69cb3b46b2a7c2907'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2022-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b16ad69cb3b46b2a7c290a'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Ravi',
    life_span: 20,
    timestamp: ISODate('2019-07-15T00:00:00.000Z'),
    expenses: 20000
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
animal> db.wild_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 55000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'banana' ],
    expenses: 70000
  },
  {
    animal_name: 'Deer',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 20000
  },
  {
    animal_name: 'Leopard',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 45000
  }
]
animal> db.domestic_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'rice' ],
    expenses: 5000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 3000
  },
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'hay' ],
    expenses: 8000
  },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'leaves', 'grass' ],
    expenses: 2500
  },
  {
    animal_name: 'Horse',
    favorite_foods: [ 'grass', 'oats' ],
    expenses: 12000
  }
]
```